Taller de Desarrollo de Software - Desarrollo Backend

Universidad de SanAndrés

Agenda

- Bloque 1: El Lenguaje de la Web
- Bloque 2: APIs RESTful y Diseño Profesional
- Bloque 3: Seguridad y Control
- Bloque 4: Arquitectura y Calidad de Código
- Bloque 5: Infraestructura

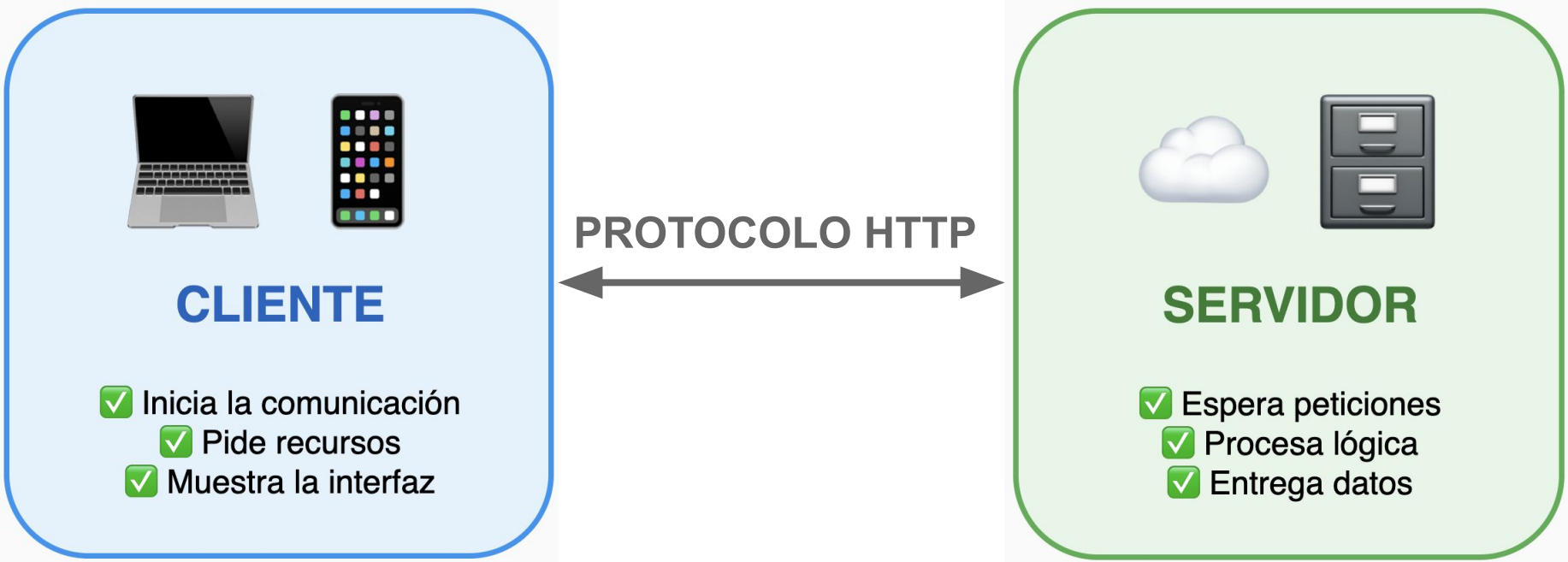
Universidad de SanAndrés

Bloque 1: El Lenguaje de la Web

- Arquitectura
- HTTP
- Puertos

Universidad de SanAndrés

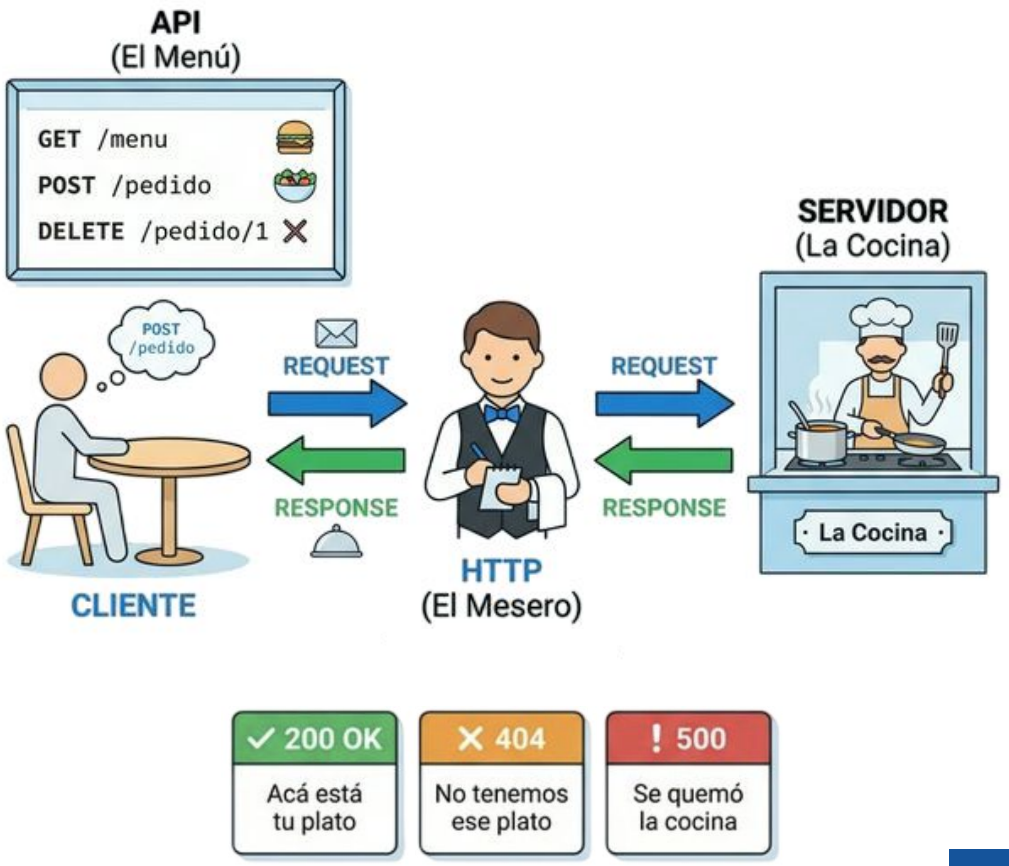
Arquitectura Cliente-Servidor



Universidad de SanAndrés

Analogía del Restaurante: Cómo funciona una Rest API

- **Cliente**: App Mobile
- **Mesero**: Protocolo HTTP
- **Menú**: API
- **Cocina**: El Servidor
- **Estado del Pedido**: Los Códigos de Estado



Universidad de SanAndrés

IP y Puertos

Para conectarte a un servicio necesitás saber dónde está (IP) y qué servicio querés (Puerto)

🌐 Dirección IP

Cada dispositivo conectado a una red tiene un **número único** que lo identifica: su **dirección IP**.
Es como el DNI de una máquina en Internet.

```
127.0.0.1 ← vos mismo (localhost)
192.168.1.x ← red local (tu wifi)
142.250.185.14 ← google.com
```

🚪 Puerto

Una misma máquina puede correr **varios servicios**. El puerto dice **a cuál querés hablar**.

Va de 0 a 65535. Los conocidos son los **bajos**.

```
# IP + Puerto = servicio específico
localhost:8080 ← tu API
localhost:5432 ← tu DB
```

Puertos más comunes:

- **:80** HTTP
- **:443** HTTPS 🔒
- **:8080** Spring Boot ☕
- **:5432** PostgreSQL 🐘
- **:3306** MySQL 🐬

Universidad de SanAndrés

HTTP: Estructura del Mensaje

HTTP REQUEST

1. Start Line (Verbo + Ruta + Versión)

```
POST /api/pedidos HTTP/1.1
```

2. Headers (Metadatos)

```
Host: api.restaurante.com
Content-Type: application/json
Authorization: Bearer xyz123
```

3. Body (El Contenido - Opcional)

```
{
  "plato": "Pizza Napolitana",
  "mesa": 5
}
```

HTTP RESPONSE

1. Status Line (Versión + Código + Texto)

```
HTTP/1.1 201 Created
```

2. Headers (Metadatos)

```
Date: Mon, 10 Mar 2026...
Content-Type: application/json
Server: Nginx
```

3. Body (La Respuesta)

```
{
  "id": 12345,
  "status": "en_preparacion",
  "eta": "15min"
}
```

Universidad de SanAndrés

Bloque 2: APIs RESTful y Diseño Profesional

- Verbos
- Status Codes
- Buenas Prácticas

Universidad de SanAndrés

API REST

Representational State Transfer

Cada recurso tiene su URL:

```
GET /api/usuarios
GET /api/usuarios/123
POST /api/usuarios
DELETE /api/usuarios/123
```

Verbos HTTP → CRUD

GET = Leer	**POST** = Crear
PUT = Actualizar	**DELETE** = Borrar

📦 Todo es un Recurso

Usuarios, productos, pedidos… cada entidad tiene su propia **URL**.

🌍 Estándar del mercado

La gran mayoría de las APIs web siguen este estilo de comunicación.

📄 Responde en JSON

```
// GET /api/usuarios/123 → 200 OK
{
"id": 123,
"nombre": "Juan Pérez",
"email": "juan@mail.com"
}
```

Universidad de SanAndrés

Anatomía de la Comunicación (JSON)

Ejemplo: Solicitar plato (GET /v1/platos/1)

200 OK

```
{
  "id": 1,
  "nombre": "Milanesa con fritas",
  "precio": 1500,
  "disponible": true,
  "vencimiento": "2026-02-10"
}
```

Respuesta Exitosa:
Devuelve los datos solicitados en formato JSON limpio y legible.

400 Bad Request

```
{
  "type": "/errors/sin-stock",
  "title": "Producto no
disponible",
  "status": 400,
  "detail": "No quedan milanesas.",
  "instance": "/v1/platos/1"
}
```

Problem Details (RFC 7807):
Explica *qué* pasó, *por qué* y *dónde* para que el cliente pueda corregirlo.

Universidad de SanAndrés

Verbos HTTP: CRUD

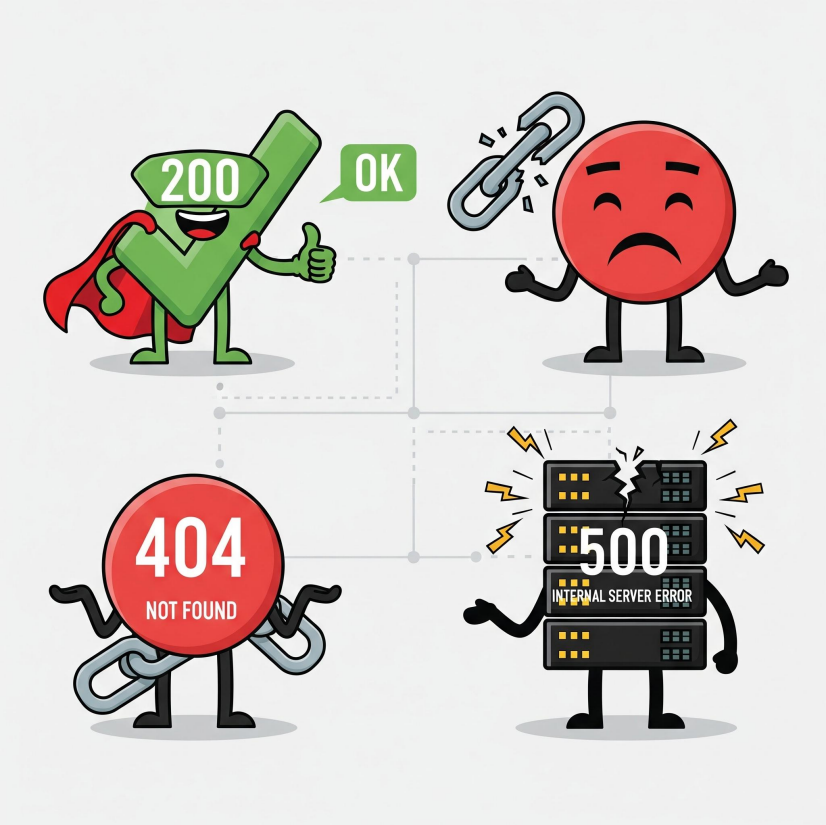
Operación	**Verbo HTTP**	**URL: /users**	**URL: /users/U123**
Create	POST	Crea un usuario	-
Read	GET	Lista todos los usuarios	Devuelve U123
Update	PUT/PATCH	Actualiza todos los usuarios	Actualiza U123 (Total / Parcial)
Delete	DELETE	Elimina todos los usuarios	Elimina U123

Universidad de SanAndrés

Códigos de Estado

Los códigos de estado indican el **resultado** de una solicitud.

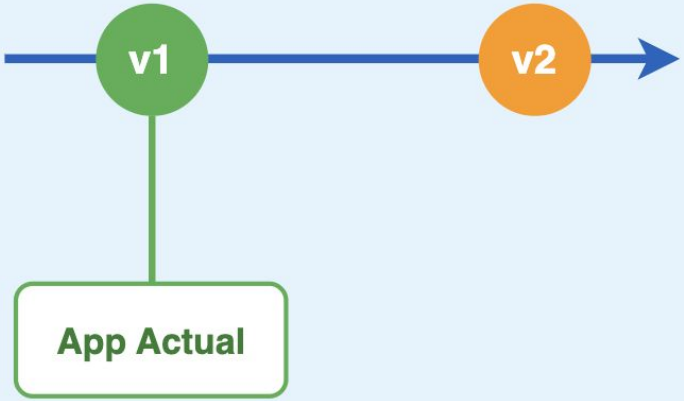
- 200 OK: La petición fue exitosa y se completó.
- 400 Bad Request: El cliente solicitó un recurso inválido.
- 401 Unauthorized: Falta autenticación para acceder al recurso.
- 403 Forbidden: El cliente no tiene permisos para esa acción.
- 404 Not Found: El recurso solicitado por el cliente no existe.
- 500 Internal Server Error: Un problema inesperado en el servidor.



Universidad de SanAndrés

Buenas Prácticas API – Evolución y Diseño

Versionado (/v1/)



Aunque la API avance a **v2**, la App sigue funcionando en **v1**.

Rutas Limpias

✅ /pending-orders

❌ ~~/getPendingOrders~~

Evita verbos (get) y camelCase en la URL.
Usa sustantivos y kebab-case.

Universidad de SanAndrés

Buenas Prácticas API - Seguridad e Idempotencia

GET: Seguro (Safe)

Garantía: Nunca modifica datos.

Si el navegador lo llama 100 veces por error, **no pasa nada malo**.

Permite hacer **CACHE** 💾

Idempotencia

Reintentos Seguros

PUT & DELETE:
Si la red falla y reintento la petición, el resultado final es **el mismo**.

PATCH no es idempotente

Universidad de SanAndrés

Bloque 3: Seguridad y Control

- CORS
- Rate Limit
- AuthN/Z
- JWT

Universidad de SanAndrés

Seguridad en la API

🌐 CORS

Cross-Origin Resource Sharing

Controla **desde qué dominios** se puede llamar a tu API.

Si tu API está en **api.miapp.com** y un sitio malicioso intenta usarla, el navegador la bloquea.

```
// En Spring Boot:
@CrossOrigin(origins = "https://miapp.com")
public class MiController {
...
}
```

✅ miapp.com → Permitido

❌ hacker.com → Bloqueado

📌 Se configura **en la API** (backend) — no en el frontend

📌 Protege al **usuario** (evita que roben sus credenciales)

⏱️ Rate Limiting

429 Too Many Requests

Limita **cuántas veces** un cliente puede llamar a tu API por minuto/hora.

Previene ataques de fuerza bruta y protege los recursos del servidor.

Ejemplo: Máximo 100 requests por minuto

```
Request #1 → 200 OK ✅
Request #50 → 200 OK ✅
Request #100 → 200 OK ✅
Request #101 → 429 Too Many Requests ⛔
```

¿Dónde se implementa?

📋 En la API (código)

🌐 Externamente (API Gateway)

📌 Protege al **servidor** (evita que se sature)

Universidad de SanAndrés

Autenticación vs Autorización

🪪 Autenticación (AuthN)

401 Unauthorized

Pregunta: ¿Quién sos?

Valida la **identidad** del usuario.

🔑 Login con usuario y contraseña
🎟️ Enviar un Token válido
📱 OAuth con Google / GitHub

📌 Mostrar el **DNI** en
la puerta del edificio

🗂️ Autorización (AuthZ)

403 Forbidden

Pregunta: ¿Qué podés hacer?

Valida los **permisos** del usuario.

👤 User → solo lee sus datos
👑 Admin → puede borrar usuarios
🚫 Nadie → borrar la DB

📌 Tener la **llave** de
una sala específica

Universidad de SanAndrés

JWT (Json Web Token) - Introducción

Un **token** es un texto que el servidor genera cuando hacés login.
El cliente lo envía en **cada request** para identificarse.

```
// En cada request, el cliente manda:
Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOjQyfQ.SflKxwRJ...
// El servidor lo decodifica y sabe:
{ "userId": 42, "role": "admin", "exp": 1700000 }
```

🚀 **Stateless**
Sin sesiones en el servidor

📦 **Auto-contenido**
La info viaja en el token

🔒 **Firmado**
Nadie puede alterarlo

⚠️ **No encriptado**
Cualquiera lee el payload

3 partes: **Header** . **Payload** . **Signature** | La firma evita que lo modifiquen, pero el contenido **no** está encriptado → **siempre usar HTTPS**

Universidad de SanAndrés

JWT - Flujo Completo

🔑 LOGIN (una sola vez)

① POST /login
Cliente envía email + password

② Valida Credenciales
Servidor chequea contra la DB

③ Genera JWT
userId + role + expiración

④ Devuelve el Token
Cliente lo guarda (localStorage / cookie)

🔄 CADA REQUEST (de ahí en más)

⑤ Manda el Token
`Authorization: Bearer eyJ...`

⑥ Verifica Firma
Lee userId/role — **sin la DB**

⑦ 200 OK
Responde con los datos

Universidad de SanAndrés

Bloque 4: Arquitectura y Calidad de Código

- Capas
- KISS
- DRY
- YAGNI

Universidad de SanAndrés

Arquitectura en Capas

❌ Sin Estructura

```
// Todo en un solo archivo 💀
app.post('/users', (req, res) => {
  if (!req.body.email) ...
  const hash = bcrypt(pass);
  db.query('INSERT ...');
  res.send(201);
});
```

Problemas:

- ❌ Un archivo hace TODO
- ❌ Imposible testear aislado
- ❌ Cambiar la DB rompe la ruta
- ❌ No se puede trabajar en equipo

✅ Arquitectura en Capas

🌐 **CONTROLLER** Rutas

⚙️ **SERVICE** Lógica

🗄️ **REPOSITORY** Datos

Ventajas:

- ✅ Cada capa: una responsabilidad
- ✅ Testeable por separado
- ✅ Cambiar DB ≠ tocar Controller
- ✅ Equipos trabajan en paralelo

Universidad de SanAndrés

Capas Controller-Service-Repository

CONTROLLER

```
@RestController
@GetMapping("/users")
Response getUsers() {
  return ok(
    service.findAll()
  );
}
```

- ✅ Mapea rutas HTTP
- ✅ Parsea parámetros
- ✅ Serializa respuestas
- ❌ NO accede a la DB

Clase: UserController

SERVICE

```
@Service
@Transactional
User createUser(dto) {
  validate(dto);
  user = map(dto);
  return repo.save(user);
}
```

- ✅ Reglas de negocio
- ✅ Orquesta repositorios
- ✅ Maneja transacciones
- ❌ NO conoce HTTP

Clase: UserService

REPOSITORY

```
@Repository
interface UserRepo
  extends JpaRepository {
  findByEmail(email);
  findByAge(min, max);
}
```

- ✅ CRUD (Create, Read...)
- ✅ Queries / Filtros
- ✅ Mapeo ORM
- ❌ NO tiene lógica

Clase: UserRepository

Universidad de SanAndrés

Principios de Diseño y Código Limpio

🧩 Separation of Concerns (SoC)

Cada capa a lo suyo.

✅ *Ejemplo:*
El **Controller** no debe saber SQL.
El **Repository** no debe saber de JSON.

💋 KISS (Keep It Simple)

La solución más simple es la mejor.

✅ *Ejemplo:*
No uses patrones complejos si un simple `if` resuelve el problema de forma clara.

♻️ DRY (Don't Repeat Yourself)

No dupliques lógica.

✅ *Ejemplo:*
Si validás el email en 3 lugares distintos, extraelo a una función común en el **Service**.

🛑 YAGNI

No programes "por si acaso".

✅ *Ejemplo:*
No crees una interfaz genérica si hoy solo tenés una implementación concreta.

Universidad de SanAndrés

Anti-Patrones vs Buenas Prácticas

❌ Lo que NO hacer

💀 Fat Controller

```
// MAL: SQL en el Controller
@PostMapping
create(dto) {
  db.query("INSERT..."); // ❌
}
```

💀 Service que conoce HTTP

```
// MAL: Service retorna HTTP
find(id) {
  return Response.status(200); // ❌
}
```

💀 Repo con Lógica

```
// MAL: Descuento en el Repo
findWithDiscount(id) {
  price = query() * 0.9; // ❌
}
```

✅ Cómo hacerlo bien

✅ Controller Delgado

```
// BIEN: Delega al Service
@PostMapping
create(dto) {
  return ok(svc.create(dto)); // ✅
}
```

✅ Service Puro

```
// BIEN: Retorna Entidad, no HTTP
find(id) {
  return repo.findById(id)
    .orElseThrow(); // ✅
}
```

✅ Repository Solo Datos

```
// BIEN: Solo datos, sin lógica
findById(id) {
  return db.query("SELECT..."); // ✅
}
```

Universidad de SanAndrés

Ciclo de vida de una request

Ejemplo: Solicitar usuario

→ Request

HTTP Request
```
GET /api/v1/users/42
```

Controller
```
@GetMapping("/{id}")
getUser(id) {
  return svc.find(id);
}
```

Service
```
find(id) {
  user = repo.findById(id);
  return user;
}
```

Repository → DB
```
findById(42)
→ SELECT * FROM users
  WHERE id = 42
```

Service valida
Recibe Entity del Repo.
Aplica reglas de negocio.
Retorna al Controller.

Controller →JSON
Recibe Entity.
Convierte a JSON.
Agrega Status 200.

← Response

HTTP Response Final
```
200 OK Content-Type: application/json
{ "id": 42, "nombre": "Fer", "email": "fgiardina@udesa.edu.ar" }
```

Universidad de SanAndrés

Bloque 5: Infraestructura

- 12-Factor
- Secrets
- Docker

Universidad de SanAndrés

Twelve-Factor App

Metodología creada por Heroku con 12 principios para construir aplicaciones modernas, escalables y mantenibles

I. Codebase
Un repo Git, múltiples deploys (dev, staging, prod)

II. Dependencies
Todo declarado en el archivo de dependencias del proyecto

III. Config
Secrets en **variables de entorno** (.env), NUNCA en el código

VI. Processes
App **sin estado** (stateless). No guardar datos en memoria entre requests

X. Dev/Prod Parity
Que tu entorno local sea **igual** al de producción (Docker)

XI. Logs
Logs a la consola (`stdout`). No escribir archivos de log

https://12factor.net/

Universidad de SanAndrés

Variables de Entorno y Secrets

Guardá los datos sensibles (**secrets**) en un archivo **.env** — nunca en el código. Tu app los lee con **System.getenv()**.

📄 **.env**

```
DB_PASSWORD=secreto123
JWT_SECRET=mi_clave_secreta
API_KEY=sk-abc123xyz
```

→

☕ **Tu código Java**

```
String pw = System.getenv("DB_PASSWORD");
// → "secreto123"
String key = System.getenv("API_KEY");
// → "sk-abc123xyz"
```

❌ **Nunca hagas esto:**

```
// Hardcodeado en el código 💀
String pw = "secreto123";
```

📌 Agregá **.env** al **.gitignore** (nunca lo pushees)
📌 Pusheá un **.env.template** (sin **secrets**)
📌 **¿Y en producción?** Cada plataforma tiene su forma de configurar las variables (Ej. CI/CD o Vault)

Universidad de SanAndrés

Qué es un Contenedor?

"Caja" con app + todo necesario para correr.

☕ Java 17 | 📚 Deps | 💾 .jar

= Tu contenedor Docker

✅ **Portátil** — funciona igual en cualquier lado

✅ **Reproducible** — siempre el mismo resultado

✅ **Ligero** — arranca en segundos

💡 **"En mi máquina funciona"**
→ ya no pasa

🐳 Dockerfile (la "receta")

```
FROM openjdk:17-slim
WORKDIR /app
COPY target/mi-api.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

⚡ Dos comandos:

```
# Construir la imagen
$ docker build -t mi-api .
# Correr el contenedor
$ docker run -p 8080:8080 mi-api
```

🟢 **Listo!** Tu app corre en localhost:8080

Universidad de SanAndrés

Docker Compose

Levanta **varios contenedores juntos** con un solo comando:

📝 docker-compose.yml → `$ docker compose up` → 🐳 **mi-api** :8080 ⟷ 🐘 **postgres** :5432

se conectan solos ✨

🐳 Servicio: api

```
api:
build: . # usa nuestro Dockerfile
ports:
- "8080:8080"
depends_on:
- db # espera a que db arranque
```

🐘 Servicio: db

```
db:
image: postgres:16 # imagen oficial
environment:
POSTGRES_DB: mi_db
POSTGRES_PASSWORD: 1234
```

Universidad de SanAndrés

Bibliografía

- Bloque 1 (Web): Computer Networking: A Top-Down Approach – Kurose & Ross (Capítulo 2: Application Layer)
- Bloque 2 (APIs): Designing Web APIs – Brenda Jin, et al. (Capítulo 2: Fundamentals of Web API Design).
- Bloque 3 (Seguridad): JSON Web Token Structure – Auth0 Docs (Link oficial).
- Bloque 4 (Arquitectura): Clean Code – Robert C. Martin (Capítulo 1: Clean Code y Capítulo 6: Objects and Data Structures)
- Bloque 5 (Infra): Beyond the Twelve-Factor App – Kevin Hoffman (Capítulo 1: One Codebase, One Repo)

Universidad de SanAndrés

Demo

Asistencia